KU LEUVEN

Natural Language Processing: POS tagging

H02B1A



Miryam de Lhoneux

Slides adapted from multiple sources

Department of Computer Science
KU Leuven

Outline

Introduction

Hidden Markov Models

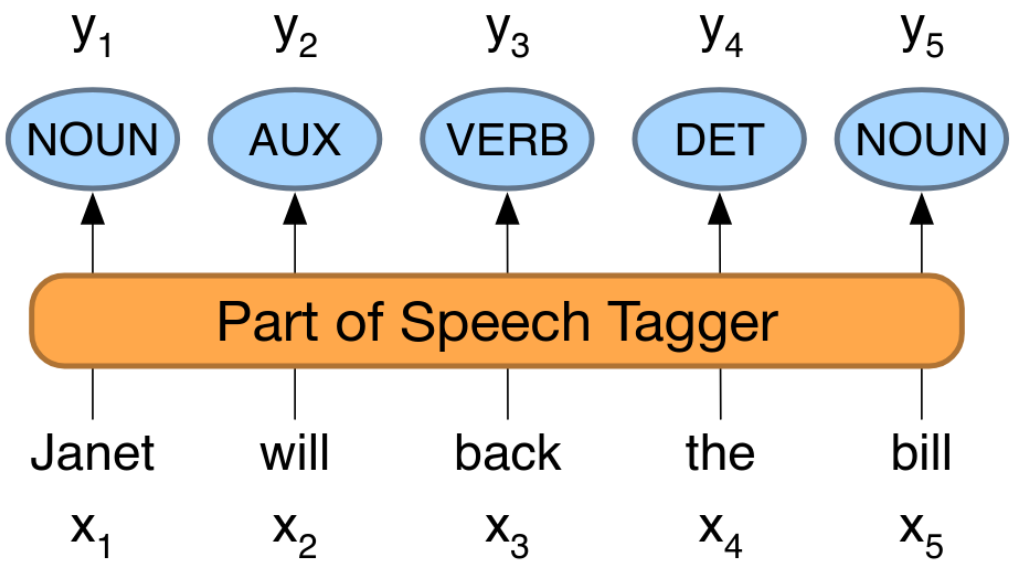
Viterbi decoding

CRF

POS tagging with neural networks

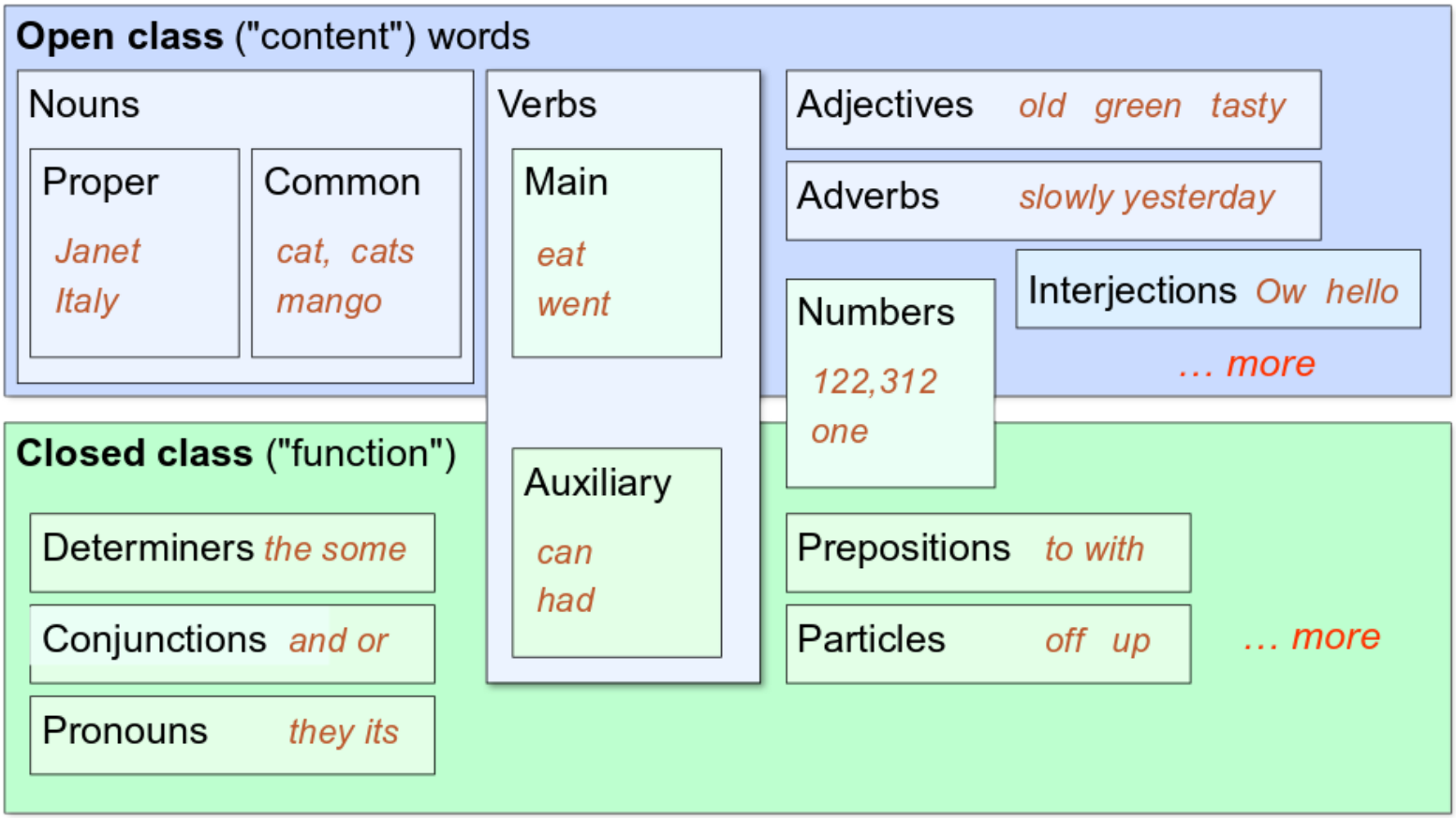
Summary and outlook

Introduction



POS tagging: mapping from input words $x_1, x_2, ..., x_n$ to output POS tags $y_1, y_2, ..., y_n$ (Jurafsky and Martin, 2025).

Introduction



KU LEUVEN

I	read	a	book
PRON	VERB	DET	NOUN

I	will	book	a	flight
PRON	AUX	VERB	DET	NOUN

Words are ambiguous and have more than one possible POS tag.

Types:		WSJ		Brown	
Unambiguous	(1 tag)	44,432	**(86%)**	45,799	**(85%)**
Ambiguous	(2+ tags)	7,025	**(14%)**	8,050	**(15%)**
Tokens:					
Unambiguous	(1 tag)	577,421	**(45%)**	384,349	**(33%)**
Ambiguous	(2+ tags)	711,780	**(55%)**	786,646	**(67%)**

Tag ambiguity in the Brown and WSJ corpora (Jurafsky and Martin, 2025)

Tag ambiguity

- Most word types are unambiguous
- But ambiguous words are very frequent

How hard is POS tagging?

- Most word types are unambiguous
- But ambiguous words are very frequent
- Still, many words are easy to disambiguate

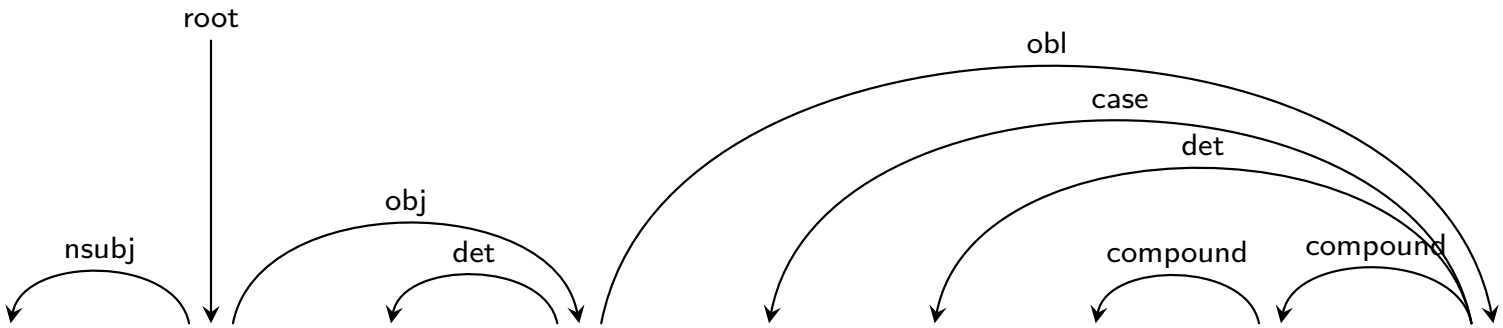
  because their different tags are not equally likely:

A	book	The	letter	A
DET	NOUN	DET	NOUN	NOUN

- Baseline: most frequent tag per word type.
- Accuracy on the WSJ: 92% vs human performance: 97%
- BUT
  - 55% whole sentence accuracy
  - Accuracy drop outside the news domain and for other languages

Why is it useful?

Can help with syntactic parsing



Who developed the algorithm for the black hole image
PRON VERB DET NOUN ADP DET ADJ NOUN NOUN

Why is it useful?

Machine translation: e.g. reordering of adjective and nouns

French			$\rightarrow$ English		
la	pomme	rouge	the	red	apple
DET	NOUN	ADJ	DET	ADJ	NOUN

Why is it useful?

Sentiment analysis: identifying adjectives

Adjective Adverb Determiner Noun Verb Preposition Pronoun

Amazing customer service | Love it | Good price | Not bad | Not an issue
Not buggy | Not happy | Not user-friendly | Not good | Low price | Low quality
Quick turn-around | Quick to fail | Inexpensive | Cheap
What is not to like about this product | I did n't get what I was promised
My package has n't arrived yet | No one in your team has been able to solve my problem
I was put on hold for an hour !



Why is it useful?

Text-to-speech: pronouncing certain words

- lead
- resume
- report

Why is it useful?

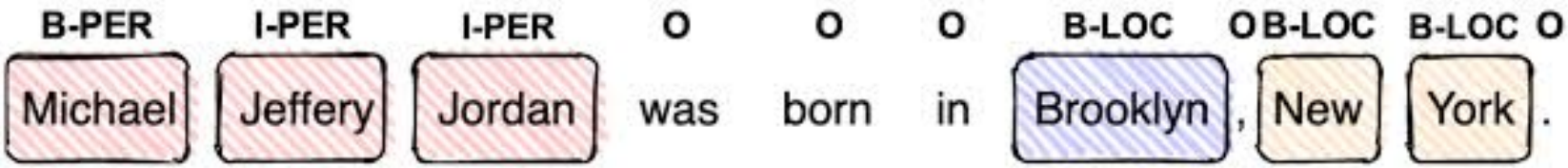
- Example of sequence tagging, a prevalent type of task in NLP
- The models and algorithms apply also to these other tasks
- Focus on POS tagging in this lecture, but see also Named Entity Recognition in Jurafsky and Martin (2025)

Named Entity Recognition (NER)



Example NER annotated text 🔗

Named Entity Recognition (NER)



Example NER annotated sentence with BIO scheme 🔗

- Treated as word-level tagging task
- Tags represent
  - if a word is a named entity
  - if it is the first word of the named entity
- This is called the BIO scheme (Begin, Interior, End), see Jurafsky and Martin (2025)

POS tagsets

- Many different tagsets
- Different levels of granularity (tagset size varies between 15 to 600)
- This lecture uses Universal Dependencies (UD):
  - A cross-linguistic standard
  - Coarse-grained: 17 tags
  - *More on UD in lecture 8*

Outline

Introduction

Hidden Markov Models

Viterbi decoding

CRF

POS tagging with neural networks

Summary and outlook

Relevant knowledge for POS tagging

- The word itself
  - Some words may only be nouns, e.g. arrow
  - Some words are ambiguous, e.g. book
  - Probabilities may help, if one tag is more likely than another
- Tags of surrounding words
  - two determiners rarely follow each other
  - two base form verbs rarely follow each other
  - determiner is almost always followed by adjective or noun

Probabilistic generative POS tagging model

To generate a sentence of length n:

Let $t_0 = <\text{s}>$

For i to n:

Choose a tag conditioned on previous tag: $P(t_i|t_{i-1})$
Choose a word conditioned on its tag: $P(w_i|t_i)$

Model assumptions

- Each tag depends only on previous tag: a bigram tag model.
- Words are independent of each other

The probability of

- a sentence $W = w_1, ..., w_n$
- and its tags $T = T_1, ..., T_n$

is defined as:

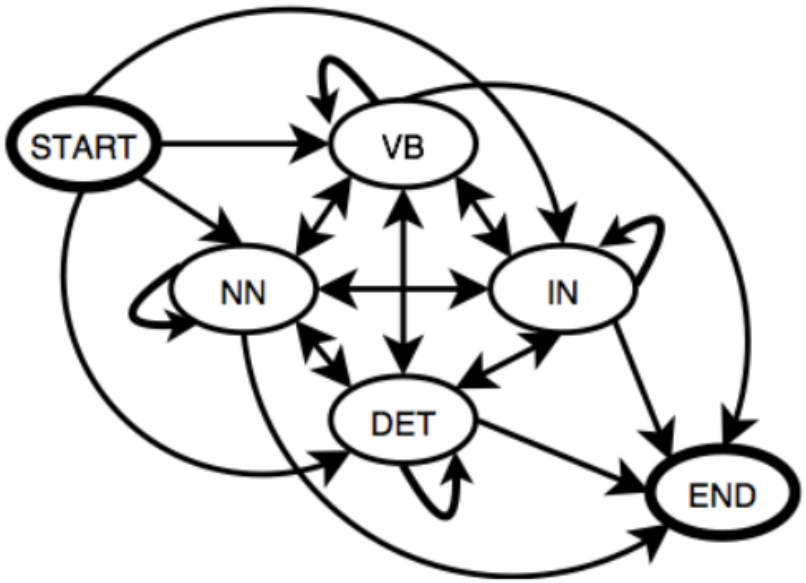
$$P(T, W) = \prod_{i=1}^{n} P(t_i|t_{i-1})P(w_i|t_i) \times P(< /s > |t_n),$$

where $w_0 = <\text{s}>, |W| = |T| = n.$

This can be thought of as a language model over words + tags

Markov chain

- One way to view the model: sentences are generated by walking through states in a graph. Each state represents a tag.
- Probability of moving from state s to s' (transition probability): $P(t_i = s' | t_{i-1} = s)$



Markov chain illustration 🔗

$t_{i-1} \backslash t_i$	PROPN	AUX	VERB	ADJ	NOUN	...
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	...
PROPN	0.3777	0.011	0.0009	0.0084	0.0584	...
AUX	0.0008	0.0002	0.7968	0.0005	0.0008	...
VERB	0.0322	0.0005	0.005	0.0837	0.0615	...
ADJ	0.0306	0.0004	0.0001	0.0733	0.4509	...
...	...	...	...	...	...	...

Probabilities estimated from the WSJ corpus (Jurafsky and Martin, 2025)

- Proper nouns often begin sentences: $P(PROPN|\text{<s>}) \approx 0.28$.
- Modal verbs (AUX) are nearly always followed by bare verbs.
- Adjectives are often followed by nouns.

$t_{i-1} \backslash t_i$	PROPN	AUX	VERB	ADJ	NOUN	...
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	...
PROPN	0.3777	0.011	0.0009	0.0084	0.0584	...
AUX	0.0008	0.0002	0.7968	0.0005	0.0008	...
VERB	0.0322	0.0005	0.005	0.0837	0.0615	...
ADJ	0.0306	0.0004	0.0001	0.0733	0.4509	...
...	...	...	...	...	...	...

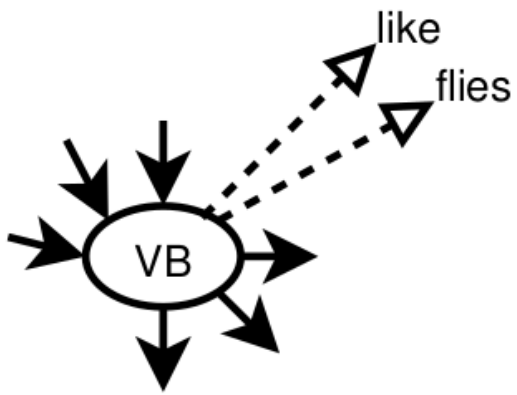
Probabilities estimated from the WSJ corpus

Notes

- This table is incomplete!
- In the full table, every row must sum up to 1 because it is a distribution over the next state (given previous).

Markov model output

- When passing through each state, emit a word
- Probability of emitting w from state s (emission or output probability): $P(w_i = w | t_i = s)$



Tagging markov model output illustration 🔗

$t_i \backslash w_i$	Janet	will	back	the	...
PROPN	0.000032	0	0	0.000048	...
AUX	0	0.308431	0	0	...
VERB	0	0.000028	0.000672	0	...
DET	0	0	0	0.506099	...
...	...	...	...	...	...

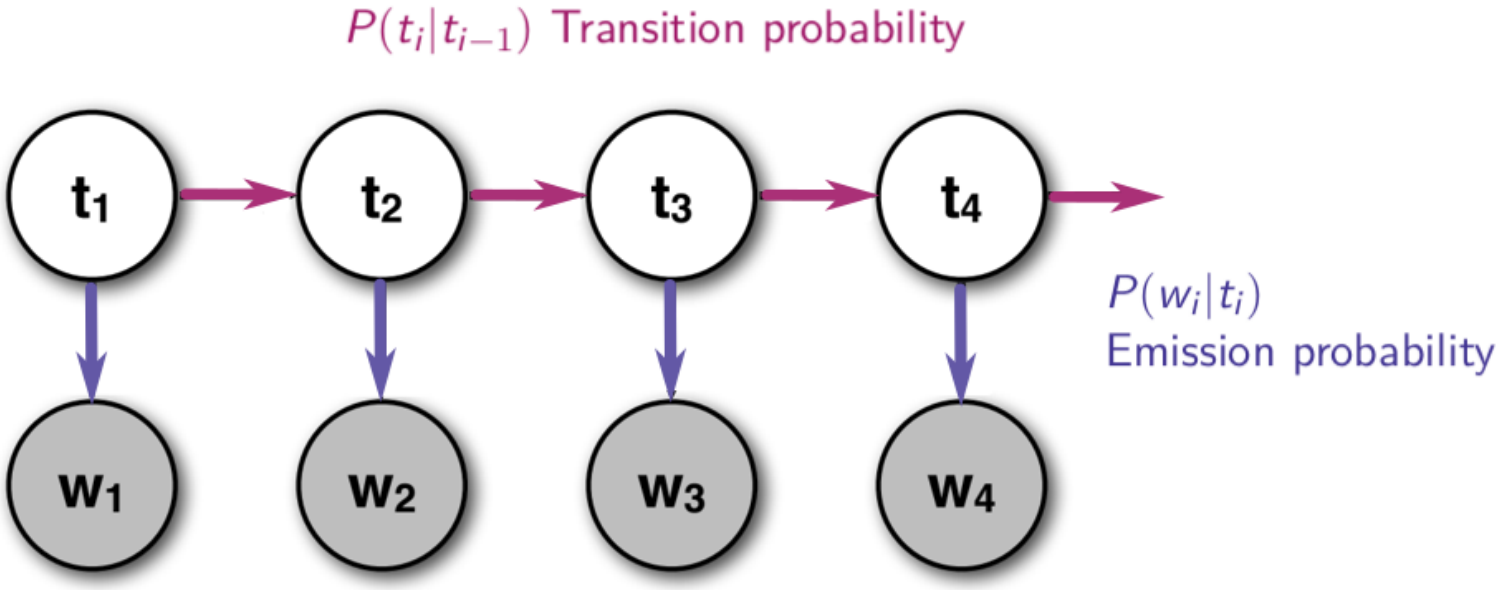
Probabilities estimated from the WSJ corpus

- 0.0032% of proper nouns are *Janet*: $P(Janet|PROPN) = 0.000032$.
- About half of determiners are *the*
- *the* can also be a proper noun. (Annotation error?)

MLE estimates

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$



HMM tagger graphical model, adapted from 🔗

Graphical model notation

- Nodes: random variables
  - Shaded: observed (here words)
  - Non-shaded: hidden (here tags)
- Arcs: causal dependency between variables

  Here: conditional probability

Using this model: example

What is the probability of this tagged sentence?

Unicorns/NOUN bring/VERB joy/NOUN

$P(NOUN|<s>)$ $P(VERB|NOUN)$ $P(NOUN|VERB)$
$P(Unicorns|NOUN)$ $P(bring|VERB)$ $P(joy|NOUN)$
$P(</s>|NOUN)$

But normally, we want to use the model to find the best tag sequence for an *untagged* sentence!

Hidden Markov Model

- hidden: at test time we only see the words/emissions; the tags/states are hidden (or latent) variables.
- Markov: Markov independence assumption (each tag/state only depends on fixed number of previous tags/states, here, just one).

Now, given a sequence of words, what is the most probable state path that generated them?

Outline

Introduction

Hidden Markov Models

Viterbi decoding

CRF

POS tagging with neural networks

Summary and outlook

Tagging or decoding

Find the best tag sequence T for an *untagged* sentence W:

$$\arg\max_T P(T|W)$$

Bayes rule:

$$P(T|W) = \frac{P(W|T)P(T)}{P(W)}$$

Drop P(W) since we are only interested in $\arg\max_T$:

$$\arg\max_T P(T|W) = \arg\max_T P(W|T)P(T)$$

$$\arg\max_T P(T|W) = \arg\max_T P(W|T)P(T)$$

- P(T) is the state transition sequence:

$$\prod_i P(t_i|t_{i-1})$$

- P(W|T) are the emission probabilities:

$$\prod_i P(w_i|t_i)$$

$$P(W|T)P(T) = \prod_i P(t_i|t_{i-1})P(w_i|t_i)$$

How do we use this?

- Goal: find the best tag sequence T^*
- Given: word sequence W

- We can compute $P(T|W)$ for any T

$\Rightarrow$ Can we just

- enumerate all possible T,
- compute their probabilites,
- choose the best one?

- Suppose we have c possible tags for each of the n words in the sentence.
- There are c^n possible tag sequences: the number grows *exponentially* in the length n

$\Rightarrow$ For all but small n, too many sequences to efficiently enumerate.

Greedy tagging

For $i = 1$ to $i = N$:

- Choose the tag that maximizes

$$P(t_i|t_{i-1})P(w_i|t_i)$$

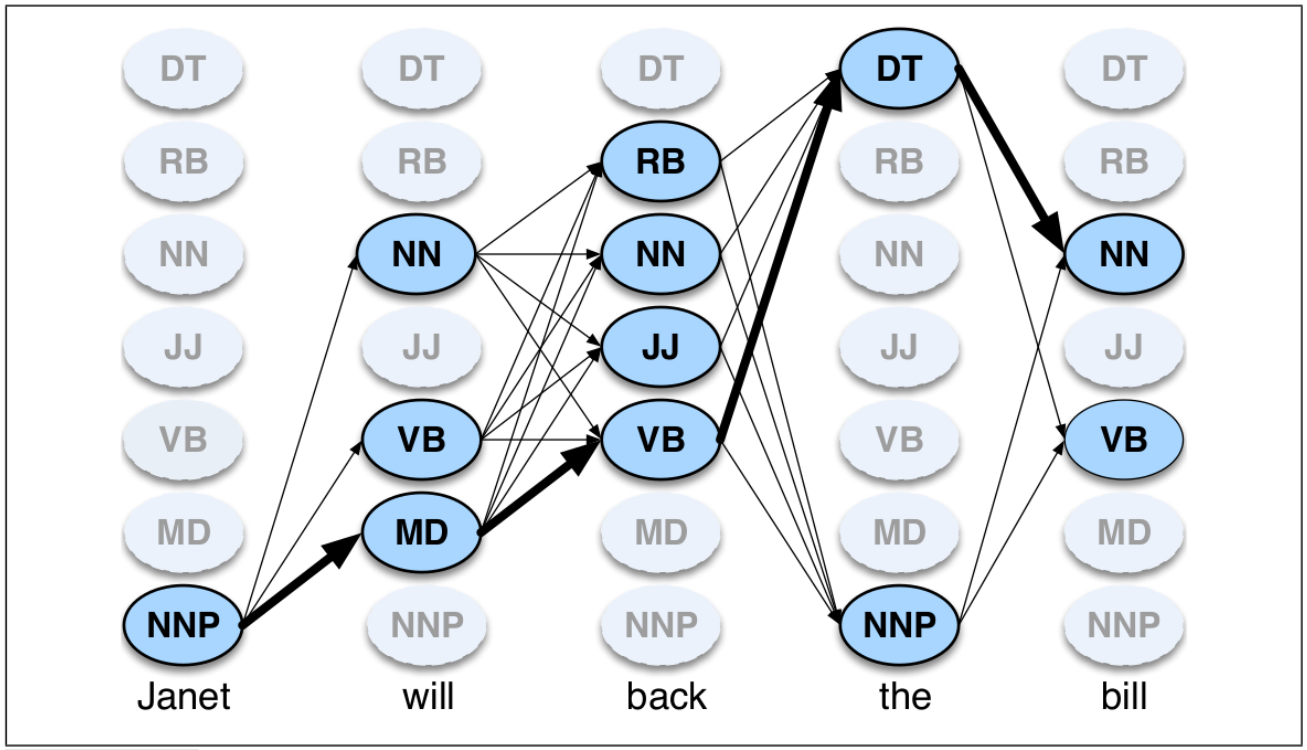
- Fast: $O(TN)$ with T tags and sentence of length N
- Problem: locally optimal solutions may lead to suboptimal global solutions.

e.g. `One bit me`

- Best tag for the word `bit` given only `one` as preceding context: *NOUN*
- Taking the full sequence into account: *VERB*

Viterbi

Viterbi decoding finds the globally optimal tag sequence using dynamic programming.



Lattice of tags with possible and best paths (Jurafsky and Martin, 2025).

	VERB	PART	NOUN	PRON
<s>	.019	.0043	.041	.67
VERB	.0038	.035	.047	.007
PART	.83	0	.00047	0
NOUN	.004	.016	.087	.0045
PRON	.23	.00079	.001	.0014

	I	want	to	race
VERB	0	.0093	0	.00012
PART	0	0	.99	0
NOUN	0	.000054	0	.00057
PRON	.37	0	0	0

	I	want	to	race
VERB	$.019 \times 0 = 0$			
PART	$.0043 \times 0 = 0$			
NOUN	$.041 \times 0 = 0$			
PRON	$.67 \times .37 = .025$			

Filling the first column

$$P(t_i|w_i) = P(t_i|\text{<s>})P(w_i|t_i)$$

	VERB	PART	NOUN	PRON
<s>	.019	.0043	.041	.67
VERB	.0038	.035	.047	.007
PART	.83	0	.00047	0
NOUN	.004	.016	.087	.0045
PRON	.23	.00079	.001	.0014

	I	want	to	race
VERB	0	.0093	0	.00012
PART	0	0	.99	0
NOUN	0	.000054	0	.00057
PRON	.37	0	0	0

	I	want	to	race
VERB	0	$.025 \times .23 \times .0093 = .000053$		
PART	0	$.025 \times .00079 \times 0 = 0$		
NOUN	0	$.025 \times .001 \times .00054 = 1.35e^{-8}$		
PRON	.025	$.025 \times .0014 \times 0 = 0$		

Filling the second column

$$P(t_i|w_i) = \max_{k=0}^{T} P(t_k|w_{1:i-1})P(t_i|t_k)P(w_i|t_i)$$

	VERB	PART	NOUN	PRON
<s>	.019	.0043	.041	.67
VERB	.0038	.035	.047	.007
PART	.83	0	.00047	0
NOUN	.004	.016	.087	.0045
PRON	.23	.00079	.001	.0014

	I	want	to	race
VERB	0	.0093	0	.00012
PART	0	0	.99	0
NOUN	0	.000054	0	.00057
PRON	.37	0	0	0

	I	want	to
VERB	0	.000053	$\max\{.000053 \times .0038 \times 0, 1.35e^{-8} \times .004 \times 0\} = 0$
PART	0	0	$\max\{.000053 \times .035 \times .99, 1.35e^{-8} \times .016 \times .99\} = .0000018$
NOUN	0	$1.35e^{-8}$	0
PRON	.025	0	0

Filling the second column

$$P(t_i|w_i) = \max_{k=0}^{T} P(t_k|w_{1:i-1})P(t_i|t_k)P(w_i|t_i)$$

	VERB	PART	NOUN	PRON
<s>	.019	.0043	.041	.67
VERB	.0038	.035	.047	.007
PART	.83	0	.00047	0
NOUN	.004	.016	.087	.0045
PRON	.23	.00079	.001	.0014

	I	want	to	race
VERB	0	.0093	0	.00012
PART	0	0	.99	0
NOUN	0	.000054	0	.00057
PRON	.37	0	0	0

	I	want	to	race
VERB	0	.000053	0	$.0000018 \times .83 \times .00012 = 1.79e^{-10}$
PART	0	0	.0000018	$.0000018 \times 0 \times 0 = 0$
NOUN	0	$1.35e^{-8}$	0	$.0000018 \times .00047 \times .00057 = 4.82e^{-13}$
PRON	.025	0	0	$.0000018 \times 0 \times 0 = 0$

Filling the second column

$$P(t_i|w_i) = \max_{k=0}^{T} P(t_k|w_{1:i-1})P(t_i|t_k)P(w_i|t_i)$$

	VERB	PART	NOUN	PRON
<s>	.019	.0043	.041	.67
VERB	.0038	.035	.047	.007
PART	.83	0	.00047	0
NOUN	.004	.016	.087	.0045
PRON	.23	.00079	.001	.0014

	I	want	to	race
VERB	0	.0093	0	.00012
PART	0	0	.99	0
NOUN	0	.000054	0	.00057
PRON	.37	0	0	0

	I	want	to	race
VERB	0	.000053	0	$1.79e^{-10}$
PART	0	0	.0000018	0
NOUN	0		0	$4.82e^{-13}$
PRON	.025	0	0	0

Follow the backtrace

PRON VERB PART VERB

HMMs in practice

- Bigram HMM model gives limited context
- Extension to higher-order N-grams
  - But this is computationally expensive
- Extensions to add arbitrary features e.g. capitalization (us=*PRON*, US=*PROPN*)
  - This is difficult
- These extensions are necessary to achieve high accuracy
- Difficulty of combining arbitrary features in a principled way is a general property of generative models

Generative vs discriminative

- Difficulty of combining arbitrary features in a principled way is a general property of generative models
- Discriminative models can do this!
- Conditional Random Fields (CRF) are discriminative sequence models

Outline

Introduction

Hidden Markov Models

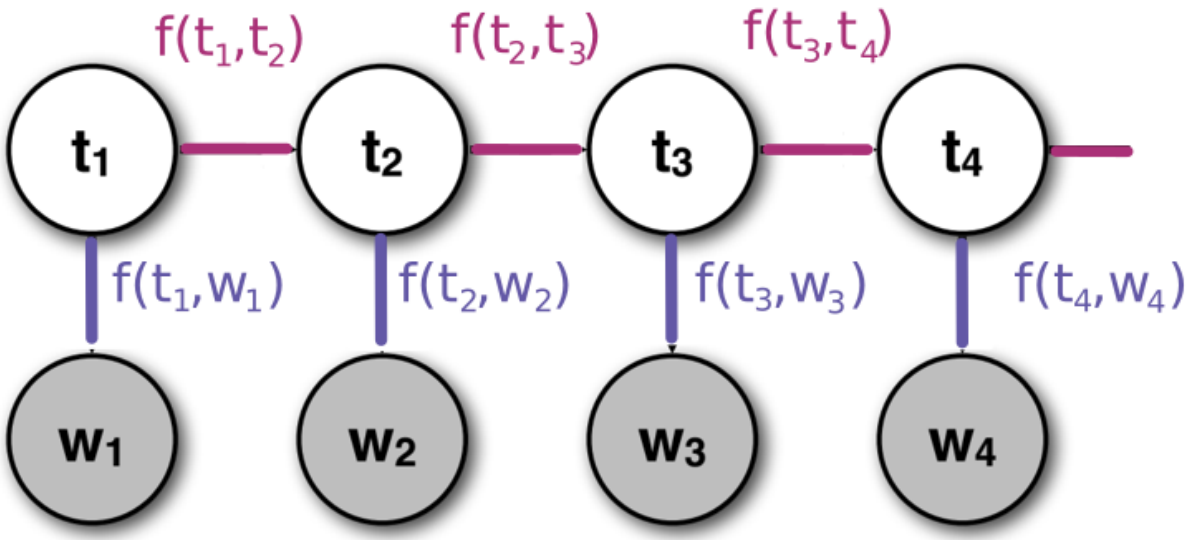
Viterbi decoding

CRF

POS tagging with neural networks

Summary and outlook

From HMM to CRF

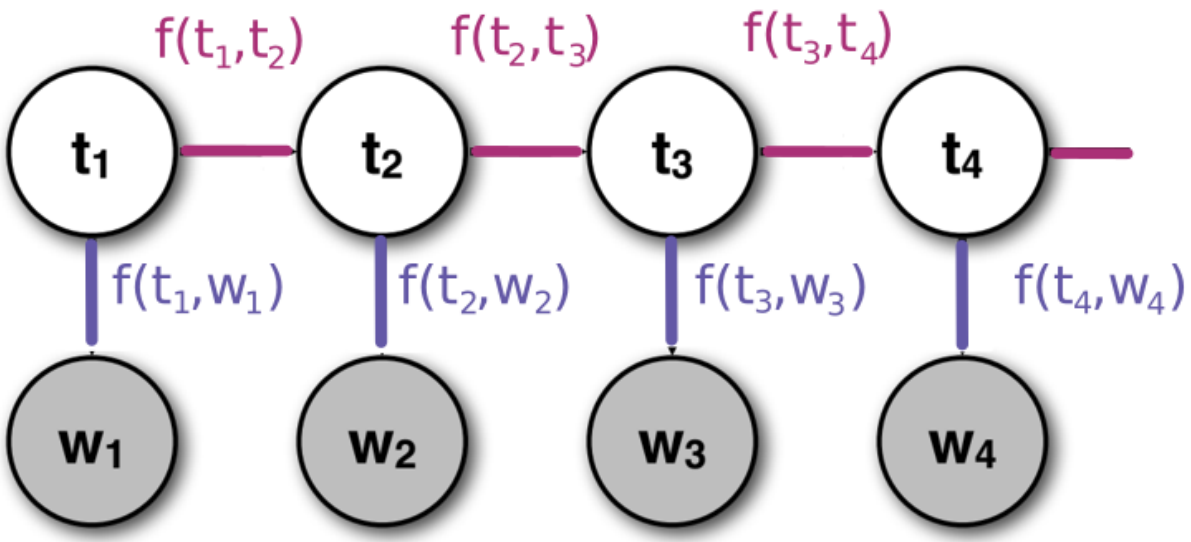


CRF tagger graphical model, adapted from 🔗

Sequential multinomial logistic regression

$$P(T|W) = \frac{exp(\sum_{k=1}^{K} w_k F_k(W, T))}{exp(\sum_{T\prime \in T} \sum_{k=1}^{K} w_k F_k(W, T\prime))}$$

From HMM to CRF

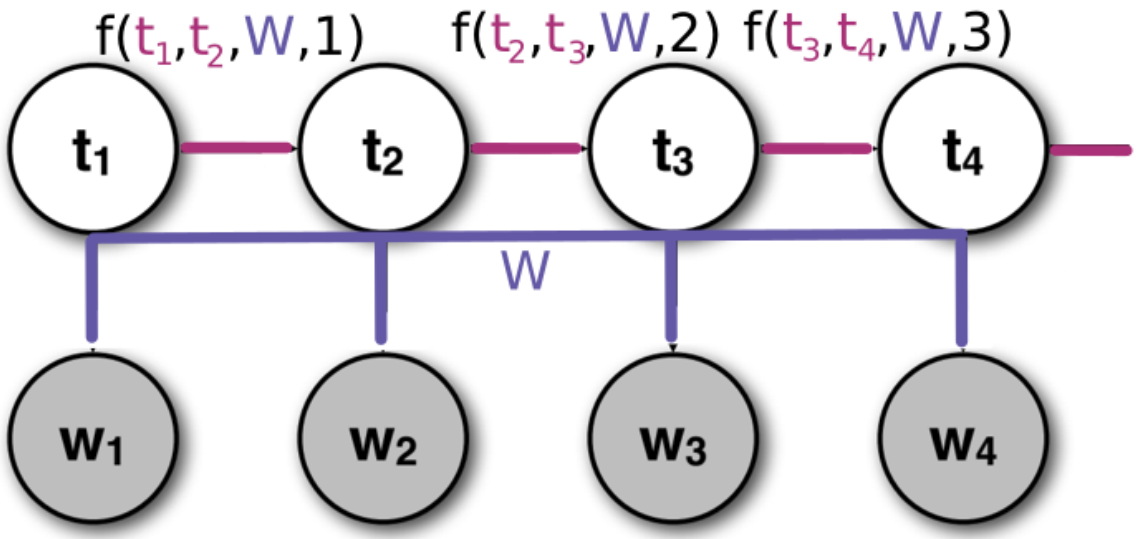


CRF tagger graphical model, adapted from 🔗

Using the same information as a bigram HMM

$$F_k(W, T) = \sum_{i=1}^{n} f_k(t_{i-1}, t_i, w_i)$$

From HMM to CRF



CRF tagger graphical model, adapted from 🔗

Extending to arbitrary features

$$F_k(W, T) = \sum_{i=1}^{n} f_k(t_{i-1}, t_i, W, i)$$

Decoding

We can still use viterbi.

We calculate the cells differently:

$$P(t_i|w_i) = \max_{k=0}^{T} P(t_k|w_{1:i-1})P(t_i|t_k)P(w_i|t_i) \qquad \text{HMM}$$

$$P(t_i|w_i) = \max_{k=0}^{T} P(t_k|w_{1:i-1})w_k f_k(t_{i-1}, t_i, W, i) \qquad \text{CRF}$$

Outline

Introduction

Hidden Markov Models

Viterbi decoding

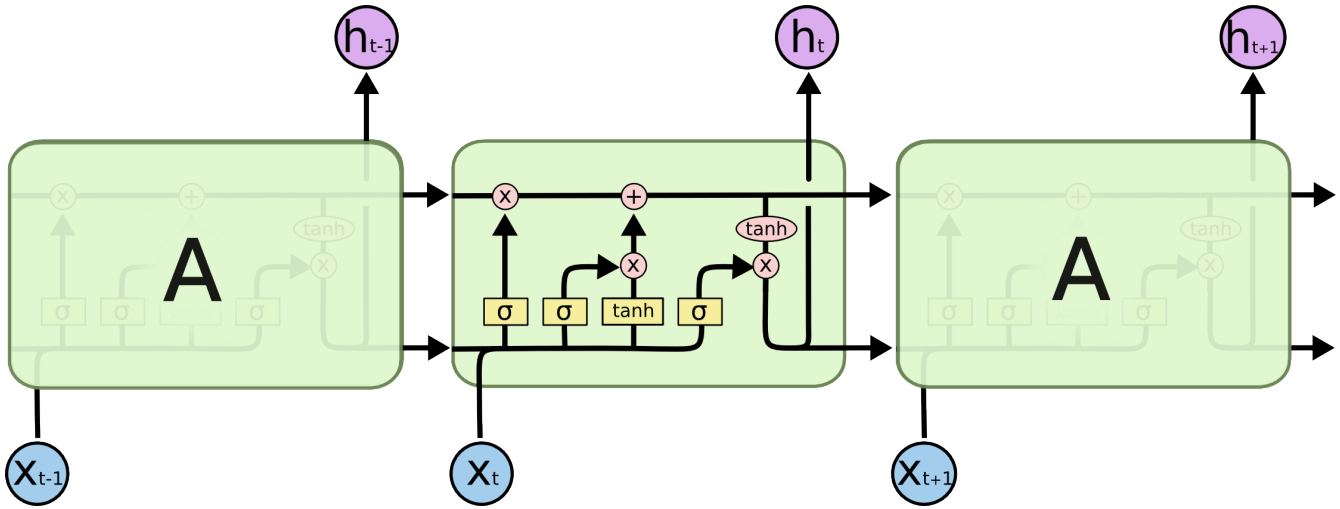
CRF

POS tagging with neural networks

Summary and outlook

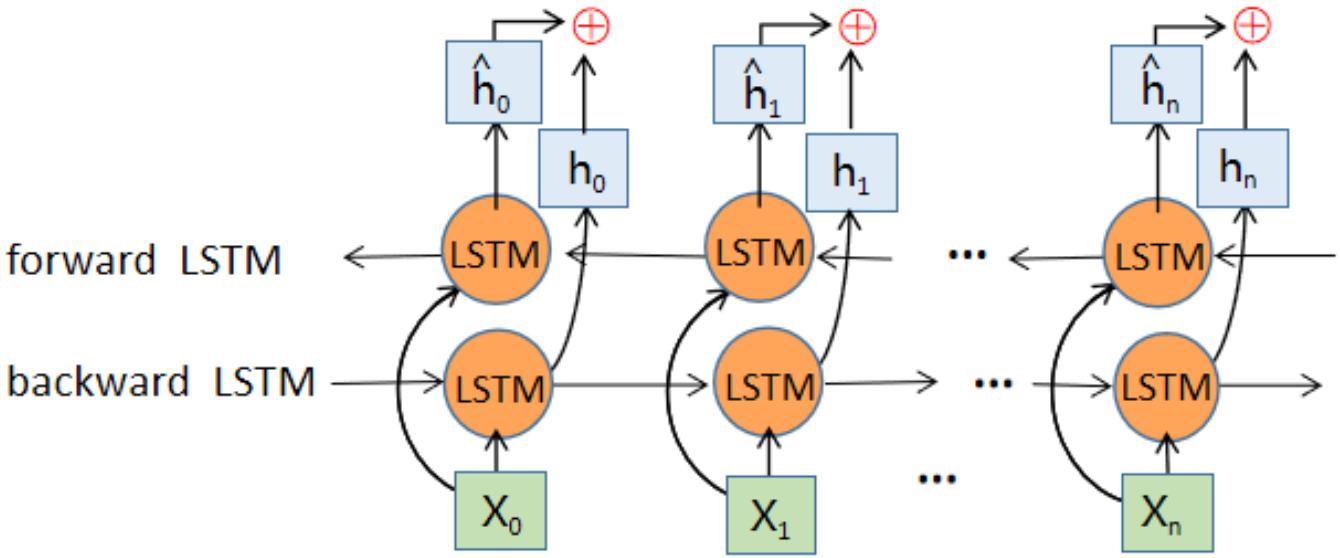
Idea: use a neural network to obtain features $f_k(W, i)$

POS tagging with LSTMs



Visual representation of an LSTM 🔗

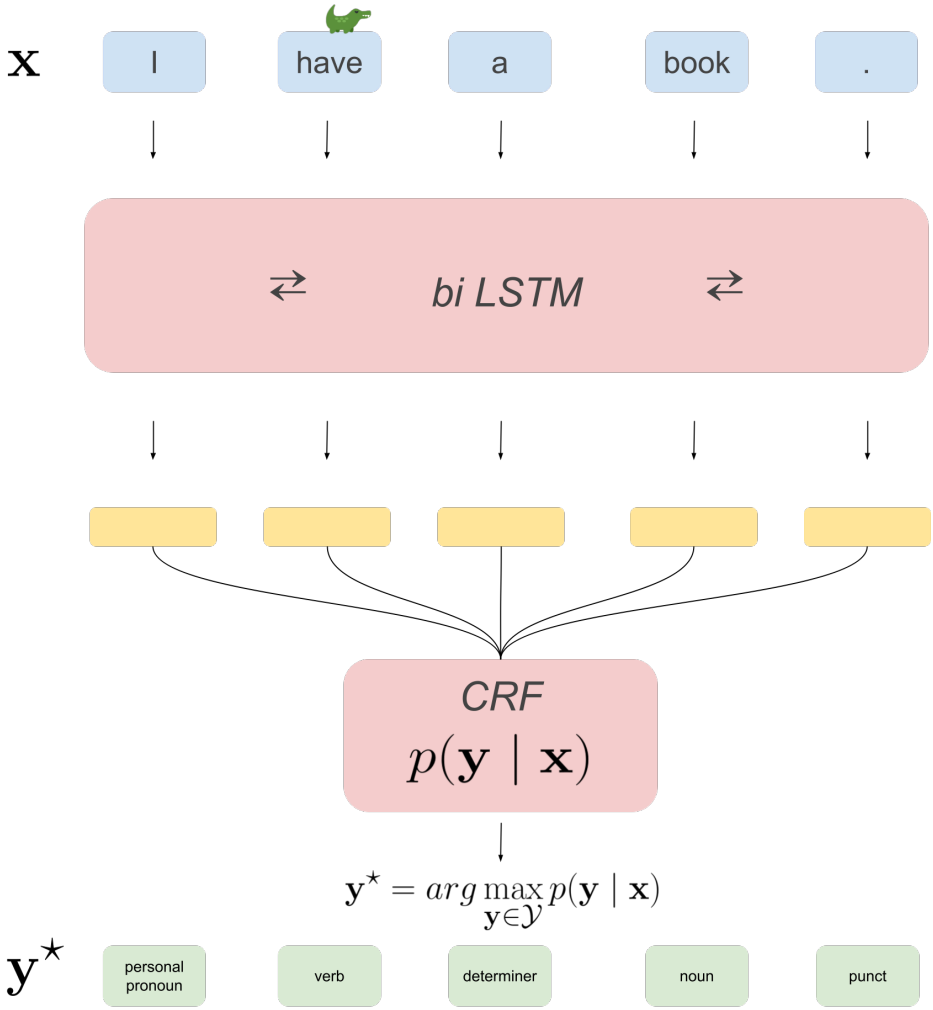
POS tagging with LSTMs



Visualization of a BiLSTM (Huang et al., 2021)

BiLSTM

Run LSTMs forward and backward through the sequence and combine the hidden nodes (here by element-wise summation, alternative: concatenation).



Visualization of a BiLSTM-CRF architecture 🔗

BiLSTM-CRF

- Output of BiLSTM is used as features for W, w_i
- CRF computes $P(t_i|w_i)$
- Viterbi is used for decoding

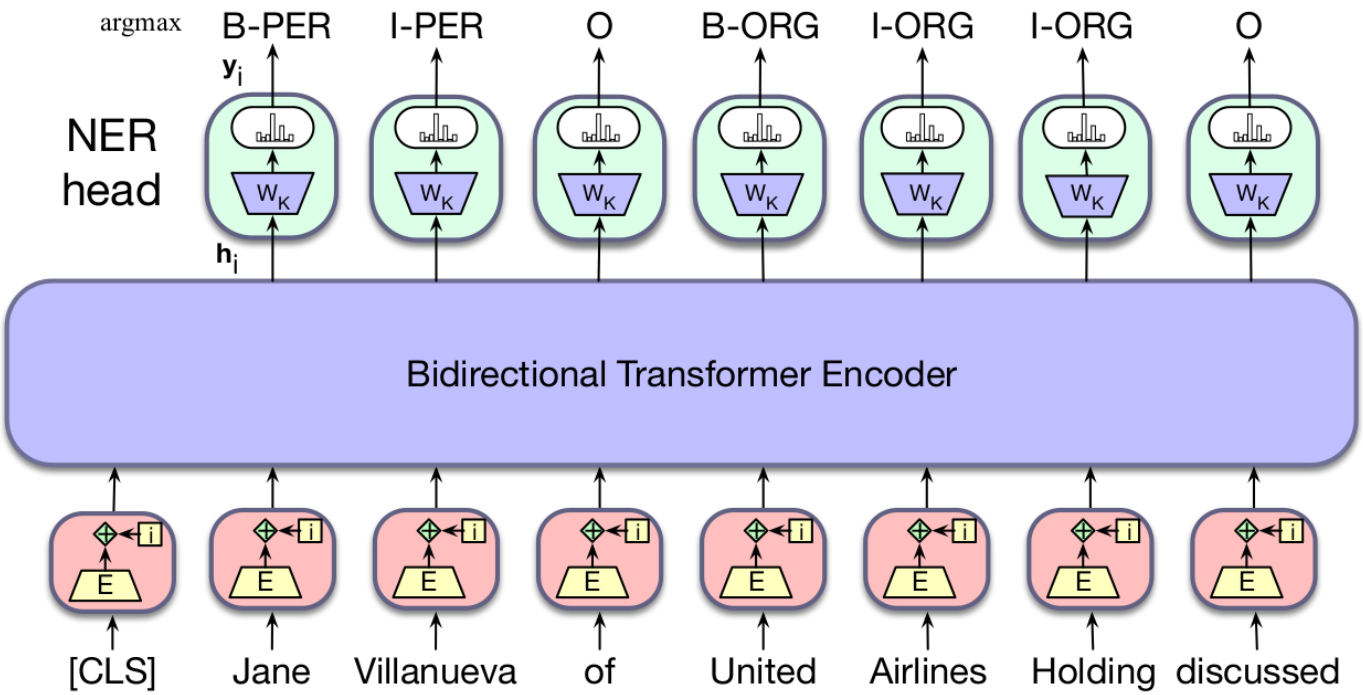
See Ma and Hovy (2016) for an example implementation.

Transformer

- Output of transformer is used as features for W, w_i
- CRF computes $P(t_i|w_i)$.
- Viterbi is used for decoding.
- Alternatively, $P(t_i|w_i)$ is computed with a softmax function (see lecture 4).
- Take the argmax tag (greedy decoding).

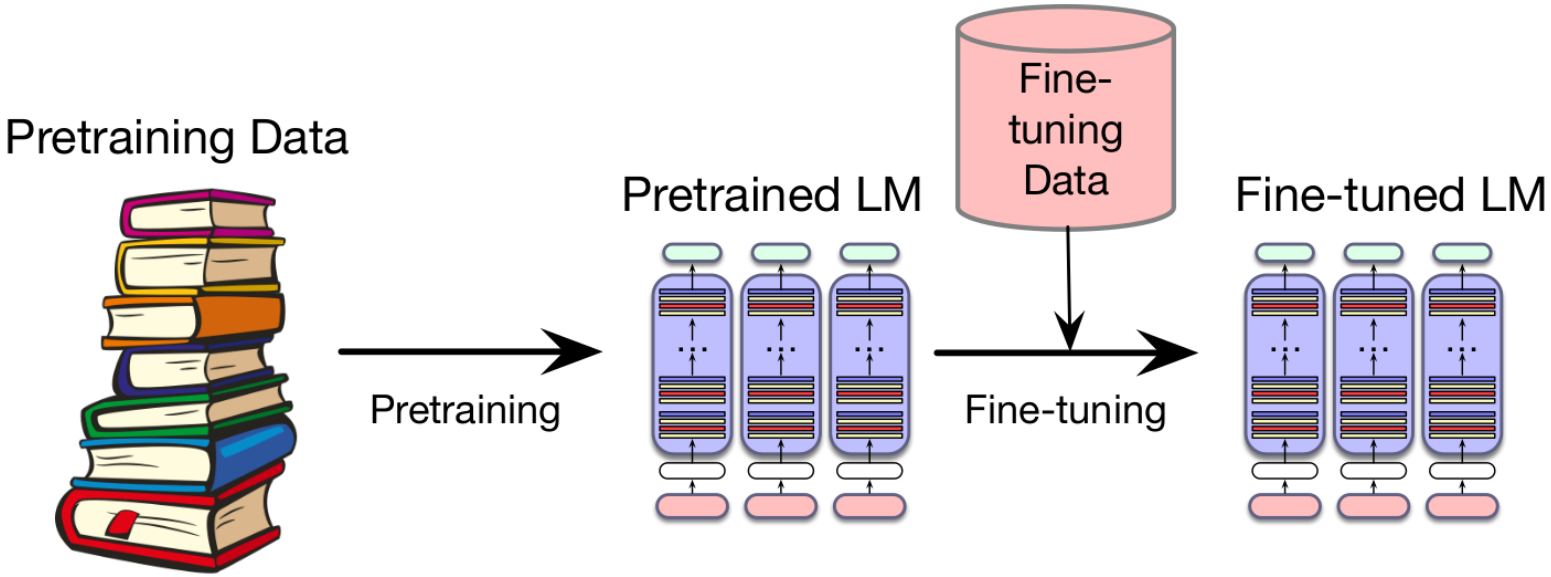
See Devlin et al. (2019) for an example implementation.

Sequential tagging with a transformer



Sequence labeling for named entity recognition with a bidirectional transformer encoder (Jurafsky and Martin, 2025)

Argmax of softmax.



Pretraining and fine-tuning (Jurafsky and Martin, 2025)

The transformer is a pretrained LM fine-tuned for the tagging task. (See lecture 4).

Evaluation and state-of-the-art

- Evaluation metric: per word accuracy
- SOTA: range between 80 to 99 depending on the language and data size
- If enough data, match human performance
- Otherwise, cross-lingual transfer may help, e.g. de Vries et al. (2022)
- If not enough data and no related language, still difficult

Outline

Introduction

Hidden Markov Models

Viterbi decoding

CRF

POS tagging with neural networks

Summary and outlook

Summary

- POS tagging: a sequential tagging task
- HMM: a generative model for POS tagging
- Decoding with viterbi
- CRF: a discriminative model for POS tagging
- BiLSTM+CRF
- Transformer with softmax and local decoding

Outlook

Evolution of tagging

- From local to global decoding
- From local to global features
- Global features of transformers are so powerful we went back to local decoding
- This is a general aspect of the evolution of NLP classification tasks

Acknowledgments

These slides take inspiration from multiple sources.
The introduction is mainly inspired by Jurafsky and Martin (2025).
The HMM slides by Nathan Schneider (based on slides by Sharon Goldwater and Philipp Koehn) 🔗
The viterbi slides by Nathan Schneider 🔗 and Mirella Lapata 🔗.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*.

Ping Huang, Luexing Zheng, Ying Wang, and Huijuan Zhu. 2021. Sentiment analysis of chinese text based on cnn-bilstm serial hybrid model. *2021 10th International Conference on Computing and Pattern Recognition*.

Daniel Jurafsky and James H. Martin. 2025. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd edition. Online manuscript released August 24, 2025.

Xuezhe Ma and Eduard Hovy. 2016. End-to-end sequence labeling via bi-directional LSTM-CNNs-CRF. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Wietse de Vries, Martijn Wieling, and Malvina Nissim. 2022. Make the best of cross-lingual transfer: Evidence from POS tagging with over 100 languages. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.